

AI usage

This file is for documenting all ai usage :D

Prompt 1 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Add a working SNMP MIB browser to the NetworkObjectSettingsWindow; make name/specs editable in the

settings menu (PC = name only, live specs; other devices = name + specs edited in software only, not pushed to the

device); improve the UI as much as possible; store the logged-in user on PC network objects; add a permission object

(user + rights) to network objects. Also mark all AI-written code with start/end comments.

****Changes made** (all AI-written regions are wrapped in `//KI start (Claude Opus 4.8, prompt 1) ...`
`//KI end`

or `<!--KI start ...--> ... <!--KI end-->`):**

- `RAT_Logic/AccessRight.cs` (new) — `[Flags] AccessRight` enum (View/Edit/Connect/Snmp/Manage/Full).
- `RAT_Logic/Permission.cs` (new) — `NetworkUser` (logic-layer user identity) + `Permission` (user + rights).
- `RAT_Logic/Session.cs` (new) — app-wide `Session.CurrentUser` so PC objects can be owned by the logged-in user.
- `RAT_Logic/MibCatalog.cs` (new) — built-in catalog of common SNMP OIDs (`MibName` + `MibCatalog.CommonNodes`).
- `RAT_Logic/NetworkObject.cs` — added stored spec fields (Os/Cpu/Gpu/Ram/Specs), `Owner`, `Permissions` list with

`SetPermission` / `RemovePermission` / `UserHasRight`, and `WalkSnmp(...)` for the MIB browser.

- `RAT_Logic/SnmpSettings.cs` — added a convenience constructor that also takes a port.
- `RAT_WPF/App.xaml` — shared dark theme brushes + styles (ModernTextBox/ModernButton/SubtleButton/DangerButton/FieldLabel).
- `RAT_WPF/Commands/LoginCommand.cs` — sets `Session.CurrentUser` on login.

- `RAT_WPF/NetworkObject_UI/NetworkObjectSettingsWindow.xaml(.cs)` — full restyle; editable Overview (PC: name only,

read-only live specs; others: name + specs in software); new **MIB Browser** tab (common-OID list + Get/Walk/Set);

working **Access Control** tab (owner + grant per-user rights).

- `RAT_WPF/NetworkObject_UI/UpdateLoginWindow.xaml`, `UpdateInterfaceWindow.xaml(.cs)`, `LoginControl.xaml(.cs)` —

restyled to match; fixed the previously dead Cancel button on the interface dialog; LoginControl now shows the real

protocol/user/port.

Notes / scope: Permissions, owner, and non-PC specs are in-memory only (no real database exists yet — DB

persistence is left as TODO). SNMP name resolution uses the built-in common-OID catalog plus raw numeric Get/Walk/Set.

Prompt 2 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Style every WPF window in our CSS brown palette (`doc/markdown/style/rats.css`); add a light/dark dropdown in

a NEW settings window built with MVVM + command binding; put the logo in the app; add anime-style transparent icons

(rat using a PC = PC icon, rat biting a cable getting electrocuted = no connection, connection-lost, device icons,

etc.); add error-handling popups (e.g. "no connection" when sending an SSH command / opening an SSH stream with no

connection); give logins a connected / not-connected status; make the interfaces list scrollable; show PC interfaces

with an interface control + an (i) info button; and filter the PC interface list down to real NICs (Wi-Fi / Ethernet /

USB-Ethernet) instead of all the garbage.

Theme / icons decision: the user asked for both vector-now and PNG-swappable-later, plus a light/dark dropdown in a

new MVVM settings window. I can't generate raster anime art, so icons are hand-built **vector** art that can be

overridden by dropping a PNG into `Assets/Icons/<key>.png` (see `IconProvider`).

Changes made (all AI regions wrapped in `KI start (Claude Opus 4.8, prompt 2) ... KI end`):

- `RAT_WPF/Themes/LightTheme.xaml`, `DarkTheme.xaml` (new) — brown palettes (light = paper+brown from `rats.css`; dark =

same fur tones on a dark base).

- `RAT_WPF/Themes/Shared.xaml` (new) — control styles (ModernButton/SubtleButton/DangerButton/IconButton/ModernTextBox/

Card/Header*) that bind brushes via **DynamicResource** so theme switches recolor live; also `AppLogo` + the icon converter.

- `RAT_WPF/Themes/Icons.xaml` (new) — vector `DrawingImage` icons: PC (rat at a computer), NoConnection (rat biting a

cable + spark), ConnectionLost, Connected, Router/Switch/Server/Client, Ethernet/Wifi/Usb.

- `RAT_WPF/Themes/ThemeManager.cs` (new) — swaps the active palette dictionary at runtime (light/dark).
- `RAT_WPF/Themes/IconProvider.cs` (new) — returns a PNG override from `Assets/Icons/<key>.png` if present, else the vector.
- `RAT_WPF/Converters/IconKeyConverter.cs` (new) — binds an icon key / device-type string to an `ImageSource`.
- `RAT_WPF/App.xaml` — composes the theme from merged dictionaries (palette swappable in slot 0).
- `RAT_WPF/SettingsWindow.xaml(.cs)` + `ViewModels/SettingsViewModel.cs` + `Commands/ChangeThemeCommand.cs` (new) —

MVVM settings window with the light/dark dropdown (bound + `ApplyThemeCommand`).

- `RAT_WPF/RAT_WPF.csproj` — bundles `Assets/logo.png` as a resource and copies `Assets/Icons/*.png` to output; added a

`RAT_Data` project reference.

- Restyled to the theme + logo: `MainWindow`, `Views/TopologyView` (logo + ⚙ Settings button), `Views/LoginView`,

`Views/NetworkObjectListingView` + `Views/NetworkObjectView` (device icons), `EnterPasswordWindow`, and the

prompt-1 windows converted to `DynamicResource` brushes.

- `RAT_Logic/NetworkObject.cs` — `IsSshConnected / IsSftpConnected / IsScpConnected / IsConnected(LoginType)` status; new

`GetOwnDeviceInterfacesDetailed()` returning real NICs only (Ethernet/Wi-Fi/USB-Ethernet), filtering loopback/tunnel/virtual.

- `RAT_Logic/HostInterfaceInfo.cs` (new) — structured host-interface model (kind/status/mac/speed/IPs).
- `RAT_WPF/NetworkObject_UI/LoginControl.xaml(.cs)` — connected/not-connected dot + label, connect with no-connection

popup, edit refresh, delete.

- `RAT_WPF/NetworkObject_UI/InterfaceControl.xaml(.cs)` (new) — host interface row with type icon, status dot, and an

(i) details popup.

- `RAT_WPF/NetworkObject_UI/NetworkObjectSettingsWindow.xaml(.cs)` — scrollable Interfaces tab; PC interfaces shown via

InterfaceControl (add-button hidden for the host); loads existing logins + delete; no-connection popups for SSH

send / SSH stream open / SFTP / SCP; shared `ShowNoConnection` / `ShowActionFailed` helpers.

Verified: full solution builds (0 errors); app launches and renders the light brown theme, the logo in the topbar,

and per-device vector icons in the sidebar.

Notes / scope: Theme switches live across all open windows because styles use `DynamicResource`. Anime PNGs are not

shipped — drop them into `Assets/Icons/<Icon.Key>.png` to override the vectors. Connection status reflects live SSH.NET

client state. The "no connection" check uses the device's open-session state, not a reachability ping.

Prompt 3 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: In dark mode the theme dropdown (and similar controls) stayed white / unreadable — fix it. Also make

windows with Apply/Close/Save buttons size to fit (or scroll) so the buttons are always reachable.
Commit as the

Visual Studio user (sumpfel), no co-author, then pull prioritising remote changes for merging — but do not push.

Changes made (AI regions wrapped in KI start (Claude Opus 4.8, prompt 3) ... KI end):

- `RAT_WPF/Themes/Shared.xaml` — fully templated, theme-aware `ComboBox` + `ComboBoxItem` (readable dropdown popup in

both themes), plus themed `ListBoxItem`, `CheckBox`, `DataGrid` / `DataGridColumnHeader` / `DataGridCell` so nothing

stays white in dark mode.

- `RAT_WPF/SettingsWindow.xaml`, `NetworkObject_UI/UpdateLoginWindow.xaml`,
`NetworkObject_UI/UpdateInterfaceWindow.xaml`

— `SizeToContent="Height"` with a `MaxHeight`, content in a `ScrollViewer`, and the button bar moved to

`DockPanel.Dock="Bottom"` so Apply/Close/Save/Cancel never scroll out of reach.

`EnterPasswordWindow` set to

`SizeToContent="Height"`. `NetworkObjectSettingsWindow` given `MinHeight/MinWidth` + window icon.

Verified: solution builds (0 errors); drove the app via UI Automation to open Settings, switch to Dark, and

confirmed by screenshot that the dropdown text ("Light"/"Dark") is legible and the whole UI recolors to dark brown.

Prompt 4 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Make the canvas dark themed and give devices a light background; default the PC name to the host's own

machine name (still changeable; other PC specs stay read-only); add a Delete tool to remove objects from the canvas.

Changes made (AI regions wrapped in KI start (Claude Opus 4.8, prompt 4) ... KI end):

- `RAT_WPF/Themes/LightTheme.xaml`, `DarkTheme.xaml` — added `Brush.Canvas` (dark in both themes), `Brush.CanvasNode`

(light) and `Brush.CanvasNodeText` .

- `RAT_WPF/Views/TopologyView.xaml(.cs)` — canvas `Background` now `Brush.Canvas` ; added a "Delete" tool

`RadioButton`; `IsDeleteToolActive` + `DeleteNode(...)` helpers.

- `RAT_WPF/Views/NetworkObjectView.xaml(.cs)` — device node now uses the light `Brush.CanvasNode` card (with border)

and shows the device **Name**; `MouseLeftButtonDown` removes the node when the Delete tool is active, and dragging is

suppressed while that tool is active.

- `RAT_WPF/ViewModels/TopologyViewModel.cs` — default PC name set to `Environment.MachineName` ; added

`RemoveNetworkObjectViewModelFromCanvas(...)` .

(Own-PC specs remaining read-only with only the name editable was already handled in prompt 1's `LoadOverview` , which

keys on `NetworkObjectType.PC` .)

Verified: solution builds (0 errors); drove the app via UI Automation + screenshots — confirmed the canvas is dark

brown, device nodes sit on light cards, and selecting the Delete tool + clicking a node removes it from the canvas.

Prompt 5 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Don't force a dark canvas / white device nodes regardless of theme — the canvas and device nodes should

follow the active theme (dark in dark mode, light in light mode).

Changes made (AI region marked `prompt 4/5`):

- `RAT_WPF/Themes/LightTheme.xaml` — `Brush.Canvas` is now light (paper-deep), `Brush.CanvasNode` light, node text dark.
- `RAT_WPF/Themes/DarkTheme.xaml` — `Brush.Canvas` stays dark, `Brush.CanvasNode` is now a dark fur tone (not white),

node text light.

Verified: built (0 errors); screenshotted both themes — light mode shows a paper canvas with light device cards,

dark mode shows a dark canvas with dark device cards; labels readable in both.

Prompt 6 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Make the canvas clearly distinct (a slightly different background and/or a border).

Changes: added `Brush.CanvasBorder` + brighter `Brush.Canvas`, and wrapped the canvas `ItemsControl` in a bordered

`Border` in `TopologyView.xaml`. (Superseded the next turn — see prompt 8.)

Prompt 7 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Improve the interfaces and logins inside the device settings window — colour the Edit button (not just

Connect/Delete), take the overall design "to the next level" (clean, useful). When adding an interface only the name

should be required, and interfaces should be editable, deletable, and show up/down via a dedicated control.

Changes made (AI regions marked prompt 7):

- `RAT_Logic/NetworkObjectInterface.cs` — added `IsUp` (software up/down state); made `IP` nullable/optional and gave

`Name` a default.

- `RAT_WPF/Themes/Shared.xaml` — new `AccentOutlineButton` style so "Edit" reads as a real (accent-outlined) action.
- `RAT_WPF/NetworkObject_UI/InterfaceControl.xaml(.cs)` — reworked into a two-mode control: read-only for host PC NICs

(icon + status pill + (i) details), and **editable** for modelled device interfaces (Toggle up/down, Edit, Delete,

(i)); raises `EditRequested` / `DeleteRequested`.

- `RAT_WPF/NetworkObject_UI/UpdateInterfaceWindow.xaml(.cs)` — only the **name** is required (IP fields optional and

only built when filled); supports **editing** an existing interface (ctor takes an optional interface, prefills).

- `RAT_WPF/NetworkObject_UI/LoginControl.xaml` — restyled into a card with a status pill and an accent-outlined Edit

button to match.

- `RAT_WPF/NetworkObject_UI/NetworkObjectSettingsWindow.xaml(.cs)` — Logins + Interfaces tabs given hints, scroll and

empty-states; software interfaces now render as editable `InterfaceControl` s wired to add/edit/delete.

Verified: built (0 errors); via UI Automation opened a seeded device's settings and screenshotted both tabs —

Interfaces show eth0 (UP, IP) and eth1 (DOWN, no IP) each with Toggle/Edit/Delete/(i); Logins show the SSH credential

with status pill and Connect/Edit/Delete. Edit is clearly coloured in both.

Prompt 8 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Remove the border around the canvas.

Changes: removed the bordered `Border` wrapper from `TopologyView.xaml` (back to a plain `ItemsControl / Canvas`)

and dropped the now-unused `Brush.CanvasBorder` from both themes. The canvas keeps its slightly-distinct background tone.

Prompt 9 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Create a window to select an interface of a device — for now it just returns the chosen interface, nothing

is done with it.

Changes made (AI region marked prompt 9):

- `RAT_WPF/NetworkObject_UI/SelectInterfaceWindow.xaml(.cs)` (new) — themed dialog that lists a device's

`NetworkInterfaces` (name + IPv4), supports double-click, and returns the picked

`NetworkObjectInterface` via the

public `SelectedInterface` property (null on cancel). Not wired to anything else yet, by request.

Verified: solution builds (0 errors).

Prompt 10 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Reworked the access-rights model (user edited `AccessRight.cs` + `User.cs`). A `NetworkObject` has multiple

`AccessRights` (one per user; a user with no entry is assumed `Hidden` / 0 — the DB already drops objects a user can't

see). `User` now has `CanCreate` (needed to create `NetworkObjects`) and a `Privileges` field. Privilege rules: a user

may change only `Hidden`/`See`/`Edit` of others if they are an **Admin** of that object; cannot take rights from another

Admin or the **Owner**; only an **Owner** can change Admins/Owners, grant/remove Owner, and delete the object.

Note on the circular reference: `AccessRight` as written referenced `User`, but `RAT_Logic` can't reference `RAT_Data`

(`RAT_Data` already depends on `RAT_Logic`). As written, `User` actually bound to `Lextm.SharpSnmpLib.Security.User`. Kept

the model in `RAT_Logic` referencing the logic-layer `NetworkUser` (extended with `CanCreate` / `Privileges`); the WPF/data

layer maps `RAT_Data.User` -> `NetworkUser`.

Changes made (AI regions marked prompt 11 / prompt 1/11):

- `RAT_Logic/AccessRight.cs` — `AccessRights` enum (`Hidden`/`See`/`Edit`/`Admin`/`Owner`) + `AccessRight { NetworkUser; AccessRights }`.

- `RAT_Logic/Permission.cs` — dropped the old `[Flags] Permission; NetworkUser` now carries `CanCreate + Privileges`.
- `RAT_Logic/NetworkObject.cs` — replaced `Owner / Permissions / SetPermission / UserHasRight` with

`List<AccessRight> AccessRights + GetRight` (Hidden default), `HasAtLeast`, `CanChangeRight` (enforces the

Admin/Owner rules), `SetRight / ApplyRight`, and `CanBeDeletedBy` (Owner only).

- `RAT_WPF/Commands/LoginCommand.cs + App.xaml.cs` — current user carries `CanCreate` (defaults true in dev until the

DB login is wired); debug-skip path seeds a dev user.

- `RAT_WPF/ViewModels/NetworkObjectViewModel.cs` — exposes `Model` so create/delete can check rights.
- `RAT_WPF/Views/TopologyView.xaml.cs` — creating (drop) requires `CanCreate` and makes the creator the Owner; the

Delete tool requires Owner (`CanBeDeletedBy`).

- `RAT_WPF/NetworkObject_UI/NetworkObjectSettingsWindow.xaml(.cs)` — Access Control tab rebuilt: shows the current

user's role, a level dropdown limited to what they may assign (Admin -> up to Edit, Owner -> any), a grid of

per-user rights, and routes changes through `SetRight` so the rules are enforced (with a clear "Not allowed" popup).

Verified: builds (0 errors); via UI Automation opened a seeded device's Access Control tab — "Your access: Owner",

grant panel enabled, grid lists debug=Owner / alice=Admin / bob=See.

Prompt 11 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: In the add-interface window, mark every IP field "(optional)" (not just IPv4) and make sure they really

are optional. Make the PC's interfaces real `NetworkObjectInterface`s on the `NetworkObject` so they can be picked in the

SelectInterfaceWindow. The SelectInterfaceWindow was buggy/weird — rebuild it nicely, e.g. reuse the InterfaceControl

without the edit/delete buttons.

Changes made (AI regions marked prompt 12 / prompt 9/12 / prompt 2/12 / prompt 4/12):

- `RAT_WPF/NetworkObject_UI/UpdateInterfaceWindow.xaml(.cs)` — every IP label now says "(optional)"; IPv6 prefix

defaults to empty; cleaned the "any IP filled?" check; only builds an `IP` when a field is filled (truly optional).

- `RAT_Logic/NetworkObject.cs` — `GetOwnDeviceInterfacesAsModel()` (real host NICs as `NetworkObjectInterface`s) and

`PopulateOwnDeviceInterfaces()` to fill the object's interface list.

- `RAT_WPF/ViewModels/TopologyViewModel.cs` — the PC palette item is populated with the host's real interfaces.
- `RAT_WPF/NetworkObject_UI/NetworkObjectSettingsWindow.xaml.cs` — own-PC tab also populates the model list if empty.
- `RAT_WPF/NetworkObject_UI/InterfaceControl.xaml.cs` — modelled-interface ctor takes `readOnly` (hides

Toggle/Edit/Delete, keeps (i)).

- `RAT_WPF/NetworkObject_UI/SelectInterfaceWindow.xaml(.cs)` — rebuilt: a styled `ListBox` whose rows are read-only

`InterfaceControl`s with a clear selection highlight; double-click or Select returns the chosen interface.

Verified: builds (0 errors); launched the SelectInterfaceWindow against the host PC and screenshotted — rows show

the real NICs (Ethernet/Wi-Fi/Local Area Connections) with status pills, the selected row is highlighted.

Prompt 12 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Make the `sshTerminalControl` not a flat black background but match the app styling, and improve the shell

styling and syntax highlighting — not just colors.

Changes made (AI regions marked `prompt 13`):

- `RAT_WPF/Themes/DarkTheme.xaml` + `LightTheme.xaml` — added a `Term.*` palette (warm dark console that stays

on-brand with the brown theme in both modes): background/input/header, text/muted, prompt, user/host/path,

command/flag/string/number, file-type colors (dir/exec/archive/image/code), and error/warn/success/border.

- `RAT_WPF/NetworkObject_UI/sshTerminalControl.xaml` — restyled into a rounded console card: header bar with

traffic-light dots + "ssh — shell #N" title, transparent themed RichTextBox with line-height, and a themed input row

with a `>` prompt glyph (no more `#0c0c0c` / `#181818`).

- `RAT_WPF/NetworkObject_UI/sshTerminalControl.xaml.cs` — replaced the ad-hoc highlighter with a tokenizing one that

resolves theme brushes and classifies: bash prompts (user/host/path/ `$` | `#`), command vs flags (`-x` / `-long`),

quoted strings, env vars (`$VAR`), unix permission strings (`drwxr-xr-x`), IPv4(:port), numbers/sizes, and files by

extension category (dir/exec/archive/image/code), plus error/warning/success log lines. Whitespace is preserved.

Verified: builds (0 errors); previewed the control with sample output and screenshotted — prompts, commands, flags,

perms, IPs, numbers, per-type file colors, and error/warn/success lines all render correctly on the themed console.

Prompt 13 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Link the C# frontend with the database (RAT-Backend). Create a `DatabaseConnection` class that implements

`IDatabaseConnection` and use it instead of the mock. Add any login WPF windows that are needed, built in the same

style as the rest of the code so it stays understandable and changeable. Don't change RAT-Backend unless necessary

(and if so, note it in the backend's `AI_usage.md`). Mark all AI-written code with start/end comments and document the

usage here.

Decisions (asked the user): wire the connection all the way into the UI (topology is loaded from the DB after

login, and create/move/delete are persisted), and add small backend endpoints where the API was missing something

(see RAT-Backend `AI_usage.md`, KI-10).

****Changes made** (all AI regions wrapped in ``//KI start` (Claude Opus 4.8, prompt: link the C# frontend with the

RAT-Backend database) `... //KI end``):**

- `RAT_Data/DatabaseConnection.cs` (new) — real `IDatabaseConnection` over HTTP to the FastAPI backend using

`HttpClient` + `System.Text.Json` (no new NuGet package; built into net9.0). OAuth2 password login (`POST /user/login`)

yields a JWT bearer token that is sent on every other call. `GetNetworkGraph()` composes the topology client-side from

`/networkObject`, `/networkObjectInterface`, `/networkObjectConnection` and `/networkObjectPermission` (the backend has

no single graph route). Per-device logins are keyed by the caller's `NetworkObjectPermission` id, which is cached per

object during the graph load. `EditUser` / `DeleteUser` throw `NotSupportedException` (no backend endpoint exists).

- `RAT_Data/UserSettings.cs` — made `Zoom` / `ShowPorts` / `ShowInterfaces` public so `EditUserSettings` can send them

(`PUT /user/settings/`); defaults mirror the backend.

- `RAT_WPF/Stores/DatabaseConnectionStore.cs` (new) — app-wide holder for the active connection, mirroring the existing

static `RAT_Logic.Session.CurrentUser` pattern (the project uses no DI container).

- `RAT_WPF/Commands/LoginCommand.cs` — the existing login screen (`LoginView/LoginViewModel` already present) now really

authenticates: builds a `DatabaseConnection` from the entered server IP/port + credentials, calls `Login()`, maps the

returned `RAT_Data.User` onto `Session.CurrentUser` (real ID / `CanCreate`), stores the connection, and navigates on

success; on failure it shows the error and stays on the login screen. No new window was needed — the styled

`LoginView/LoginViewModel/LoginCommand` MVVM trio already existed, so it was wired up rather than rebuilt.

- `RAT_WPF/App.xaml.cs` — `_debugging_ignore_login` now defaults to `false` so the real (DB-backed) login screen is shown

(flip back to `true` to skip login during development; the canvas then starts empty and saving is a no-op).

- `RAT_WPF/ViewModels/TopologyViewModel.cs` — after construction, `LoadFromDatabaseAsync()` loads the saved graph from the

connection and puts each device on the canvas (no-op + empty canvas when there is no connection).

- `RAT_WPF/Views/TopologyView.xaml.cs` — dropping a new device persists it (`AddNetworkObject`, which also makes the

creator the Owner server-side and assigns the real id); moving a saved device persists its X/Y (`EditNetworkObject`);

the Delete tool deletes it on the server first and only then removes it from the canvas (keeps the node if the server

refuses). All persistence failures surface via a `MessageBox`; everything is a no-op without a connection.

Backend (necessary, documented in `RAT-Backend/doc/markdown/AI_usage.md`, KI-10): added `GET /user/` (list users) and

exposed `canCreate` on the user DTOs so the client can resolve permission rows to usernames and know who may create;

fixed `NetworkObjectInterfaceOut.network_object_connection_id` to be `Optional[int]` (it was a non-optional `int` but is

NULL for unconnected interfaces, which crashed `GET /networkObjectInterface/` and blocked the client's graph load).

Verified: `RAT_Data` + `RAT_Logic` build with 0 errors on Linux (the WPF project targets `net9.0-windows` and can

only be built on Windows — the user will verify the full UI build there). The backend flow the client depends on was

smoke-tested against a throwaway DB via the running server: form login → JWT, `/user/me` and `/user/` return

`can_create`, `NetworkObject` create/list, permission row (creator = Owner), per-device login create/list, user-settings

PUT, and — after the interface fix — interface and connection create/list all return shapes matching the client DTOs.

Prompt 14 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: The client only managed to *create* things; editing/deleting `NetworkObjects`, and anything to do with

interfaces / connections / logins, did nothing. Fix all of that so it persists. Make the windows a bit larger and

prettier. Add a way (admin only) to create other users. Make granting permissions for a `NetworkObject` work perfectly.

Treat a PC as a normal `NetworkObject` for everyone else (just shown as a "PC"); only the owner(s) of their own machine

see live PC interfaces/stats. Mark everything as AI, edit the backend if needed.

Decisions (asked the user): own-PC live view = current user is Owner AND the object's name matches this machine's

name; expose user-creation as an admin-only "Users" button on the topology top bar.

Root cause: `IDatabaseConnection` had no methods for interfaces, connections or permissions, and the settings

window did everything in memory (`// TODO: Save to Database`). Permissions used a fake ``NetworkUser(name,`

`name.GetHashCode())``, so they could never map to a real account.

Changes made (all AI regions wrapped in `//KI start (Claude Opus 4.8, prompt 14) ... //KI end`):

- `RAT_Logic/NetworkObjectInterface.cs` — added `ID` + `NetworkObjectId` (db identity / owning object) so an interface

can be edited/deleted on the backend.

- `RAT_Logic/NetworkConnection.cs` — added `ID` (db identity) so a connection can be deleted.
- `RAT_Logic/AccessRight.cs` — added `ID` (backend permission-row id) so a permission can be deleted.
- `RAT_Data/IDatabaseConnection.cs` — added interface CRUD (`AddInterface` / `EditInterface` / `DeleteInterface`),

connection CRUD (`AddConnection` / `DeleteConnection`) and permissions

(`GetNetworkObjectPermissions` / `SetPermission` /

`DeletePermission`).

- `RAT_Data/DatabaseConnection.cs` — implemented all of the above over the matching backend routes; `GetNetworkGraph`

now fills interface/connection IDs, loads all users, and attaches each object's real `AccessRights` (resolved to

usernames). `SetPermission(Hidden)` is routed to a delete (`Hidden == no row`).

- `RAT_Data/DatabaseConnectionMock.cs` — stubs for the new interface methods so it still compiles.
- `RAT_WPF/NetworkObject_UI/NetworkObjectSettingsWindow.xaml(.cs)` — every action now persists through the connection:

Save overview (`EditNetworkObject` / `AddNetworkObject`), add/edit/delete login, add/edit/delete interface, and the

Access Control tab loads users + rights from the DB, grants via `SetPermission` against a **real** user picked from a

dropdown (was a free-text box), and shows the live rights grid. The window is larger (900×720) and centered.

- `RAT_WPF/NetworkObject_UI/LoginControl.xaml.cs` — added an `Edited` event so the parent can persist a login edit

(and the db id is preserved across the edit).

- `RAT_WPF/ViewModels/TopologyViewModel.cs` — creating a connection now persists it (`AddConnection`), requiring both

interfaces to be saved first (clear popup otherwise).

- `RAT_WPF/ManageUsersWindow.xaml(.cs)` (new) — admin-only window that lists users and creates new ones

(username/password/admin/can-create) via `AddUser`; opened from a new  **Users** button on the topology top bar

that is only visible to admins (`Privileges >= 100`).

- `RAT_WPF/Views/TopologyView.xaml(.cs)` — the admin-only Users button + handler.
- **Own-PC rule:** `NetworkObjectSettingsWindow.ComputeIsOwnPc()` now returns true only for a PC the current user owns

whose name matches this machine; everyone else sees the stored DB fields with just the PC icon (no live host specs).

- Bigger/centered windows: `MainWindow` (1180×720), `UpdateInterfaceWindow`, `UpdateLoginWindow`, `SelectInterfaceWindow`.

Backend (necessary, documented in `RAT-Backend/doc/markdown/AI_usage.md`, KI-11):

deleting a `NetworkObject` now

cascade-deletes its interfaces, the connections those interfaces used, and the logins/snmp settings on its permission

rows — otherwise dangling rows broke the next graph load.

Verified: the **full WPF solution builds with 0 errors** (built on Linux with `-p:EnableWindowsTargeting=true`).

Backend flows smoke-tested against a throwaway DB: create user (admin), list users, create/edit interface (PUT 200),

create connection, grant a permission to another user, and delete a `NetworkObject` — confirming its interface and

connection are gone (cascade) while an unrelated object's interface stays.

Prompt 15 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Global admins don't have all permissions everywhere like they should. Deleting `NetworkObjects` doesn't

work, and connections aren't saved/loaded correctly (load each object's connections from the DB). Add more rat-themed

icons (connection lost, successfully logged in, ...). Add a Logout button that returns to the login window, remembering

the backend IP/port so only username + password are asked again. In the NetworkObject Access Control tab the names

dropdown text is invisible. Mark everything as AI; edit the backend if needed.

Changes made (all AI regions wrapped in `//KI start (Claude Opus 4.8, prompt 15) ... //KI end`):

- **Global admins now own everything (client side):** `RAT_Logic/NetworkObject.cs` — `GetRight()` returns `Owner`

for any user with `Privileges >= 100` (the backend `is_admin` flag, already enforced server-side). Since

`CanChangeRight` / `CanBeDeletedBy` / `HasAtLeast` all go through `GetRight`, admins can now manage and delete any

object even with no explicit permission row. (Verified server-side: an admin with no perm row can PUT and DELETE

another user's object — 200/200.)

- **Delete actually works:** the Delete tool used `NetworkObjectDeleteCommand`, which only removed the node from the

canvas in memory (no DB call, no owner check) — the delete was lost on reload. It now calls a new

`TopologyViewModel.DeleteNetworkObjectFromCanvasAndDatabase()` that checks ownership, deletes on the backend, then

removes the node. Removed the dead `DeleteNode` / `PersistDeletedNetworkObject` in `TopologyView.xaml.cs`.

- **Connections load + draw:** `TopologyViewModel.LoadFromDatabaseAsync()` previously added only devices. It now also

rebuilds a `NetworkConnectionViewModel` for every distinct connection in the loaded graph (each

`interface.Connection`), so saved cables are drawn after login. (`GetNetworkGraph` already reconstructs the

`NetworkConnection`s and sets them on both endpoint interfaces.)

- **Logout:** new `RAT_WPF/Commands/LogoutCommand.cs` + `TopologyViewModel.Logout()` clear `Session.CurrentUser` and

`DatabaseConnectionStore.Current` and navigate back to a fresh `LoginViewModel`. The topology now takes the

`NavigationStore` (new ctor; the parameterless one still works for the debug path). A ☺ **Logout** button (with the

new rat icon) sits on the top bar. `DatabaseConnectionStore` remembers

`LastServerIp / LastServerPort ; LoginCommand`

saves them on success and `LoginViewModel` pre-fills them, so a re-login only needs username + password.

- **Dropdown text fix:** `RAT_WPF/Themes/Shared.xaml` — the themed `ComboBoxItem` template's `ContentPresenter` now

forces `TextElement.Foreground="{DynamicResource Brush.Text}"`. `DisplayMemberPath` items (the user dropdown in

Access Control) were rendered with no inherited colour and were invisible.

- **More rat icons:** `RAT_WPF/Themes/Icons.xaml` — new vector `Icon.LoginSuccess` (happy rat + green check),

`Icon.LoginFailed` (rat + red X) and `Icon.Logout` (rat scurrying out a door); registered in `IconProvider`. The

login screen shows the happy/sad rat + a message via new

`LoginViewModel.StatusIcon / StatusMessage / ShowStatus`

(set by `LoginCommand` instead of the old failure `MessageBox`); the logout button uses the rat door icon.

Backend: no change needed — the API already grants global admins `OWNER` everywhere (`permissions.py`) and

cascade-deletes an object's interfaces/connections (KI-11). Verified both against a throwaway DB.

Verified: the **full WPF solution builds with 0 errors** (`-p:EnableWindowsTargeting=true` on Linux). The

admin-can-edit/delete-others and cascade behaviours were confirmed against the live backend on a throwaway DB. The

UI behaviour itself (dropdown legibility, logout flow, drawn connections, rat status icons) is for the user to confirm

on Windows.

Prompt 16 — Claude (model: Claude Opus 4.8, via Claude Code)

Request (bug list): (1) on a 500 the device must NOT be added to the canvas; (2) drag & drop must work when the

press starts on a device icon (clicking "through" the image); (3) you can't see the user in the Access Control

dropdown; (4) there must be a way to edit users (change password / username / admin / can_create) — admins may edit

everyone, a normal user only themselves and only name + password (not admin/can_create); (5) opening a NetworkObject

as an admin errors out because the admin can act without a permission row, which then breaks logins. Mark everything

as AI; edit the backend if needed.

Changes made (all AI regions wrapped in `//KI ... (prompt 16)`):

- **(1) No phantom device on error:** `RAT_WPF/Views/TopologyView.xaml.cs` — `PersistNewNetworkObject` now returns

`Task<bool>` and `NetworkObject_Drop` is `async`: the device is added to the canvas **only after** the backend

accepts it. On a 500 (or any error) the popup shows and the node is not added.

- **(2) Drag over icons:** `RAT_WPF/Views/NetworkObjectView.xaml` + `NetworkObjectListingView.xaml` — the node/palette

row got `Background="Transparent"` (whole area hit-testable) and the icon `Image` got `IsHitTestVisible="False"`, so

a press on a transparent part of the icon no longer "falls through" and the drag `MouseMove` always fires.

- **(3) Dropdown text:** `RAT_WPF/NetworkObject_UI/NetworkObjectSettingsWindow.xaml(.cs)` — the user `ComboBox` now uses

an explicit `ItemTemplate` (a `TextBlock` bound with `Foreground="{DynamicResource Brush.Text}"`) instead of

`DisplayMemberPath`, so the names are visible in both the dropdown and the closed selection box.

- **(4) Edit users:** new reusable `RAT_WPF/EditUserWindow.xaml(.cs)` dialog (username + password + optional

Admin/Can-create, hidden when the caller may not change them).

`RAT_Data/DatabaseConnection.EditUser` now PUTs to

`/user/{id}` (empty password = unchanged). Admin path: an **Edit** button per row in `ManageUsersWindow` (all fields).

Self path: an **Account** → **Edit my account** section in the app `SettingsWindow` (everyone), with admin fields shown

only to admins; the in-memory `Session.CurrentUser` is updated on save. The backend enforces the real rules.

- **(5) Admin-without-permission-row logins:** `RAT_Data/DatabaseConnection` — `GetMyPermissionId` was split into

`TryGetMyPermissionId` (returns null instead of throwing → reads just show no logins) and `EnsureMyPermissionId`

(creates the row by self-granting `See` when missing, which a global admin/Admin/Owner is allowed to do).

`GetUserDeviceLogin` no longer throws for an admin with no row; `AddUserDeviceLogin` lazily creates the row.

Backend (necessary, documented in `RAT-Backend/doc/markdown/AI_usage.md`, KI-12): added PUT `/user/{id}` with a

`UserEdit` model — a global admin may change any user's username/password/is_admin/can_create; a normal user may edit

only themselves and only username + password (is_admin/can_create are ignored for a non-admin self-edit; editing

another user is 403). Username uniqueness is enforced.

Verified: full WPF solution builds with 0 errors (`-p:EnableWindowsTargeting=true`). Backend smoke-tested

against a throwaway DB: admin edits another user (rename + promote, 200); a normal user self-edits name+password (200,

re-login works); a normal user editing someone else is **403**; a normal user trying to self-grant is_admin/can_create

is silently ignored (stays non-admin); and an admin with no permission row self-grants `See` (201) then adds a login

(201) — the previously-crashing path. UI behaviour (no phantom device, drag over icons, dropdown legibility, the edit

dialogs) is for the user to confirm on Windows.

Prompt 17 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: (1) The Access Control `PermUserCombo` binding (`{Binding UserName}`) didn't show anything. (2) Better PC

logic: a new PC should have normal default stats (not blank), changing only the name shouldn't wipe the other specs,

and a freshly dragged PC should keep the same settings as a previously deleted one. (3) Add an About window crediting

Tobias and Claude with his helper Christof, phrased fancy, with a cool rat-themed vector graphic. (4) Give the

"couldn't talk to the database" popups rat-themed vector graphics too.

Changes made (AI regions marked `prompt 17`):

- **Combo binding fix** — `NetworkUser.UserName/ID/CanCreate/Privileges` and `RAT_Data.User.UserName/Password/ID` were

public **fields**; WPF only binds to **properties**, so the combo's `{Binding UserName}` silently showed blank rows.

Converted them to auto-properties (source-compatible). `PermUserCombo` now shows the user names.

- **PC specs** — `NetworkObject.ApplyDefaultPcSpecsIfEmpty()` fills empty Os/Cpu/Gpu/Ram from this machine's real

values (never clobbers edits). Called when the PC palette item is created and in `LoadOverview` for any PC, so a new

or renamed PC always shows real stats instead of blanks. `SaveOverview` now persists the spec values for every

device (incl. the live own-PC) so they survive a rename that flips it out of the live view. (Dragging the same

palette PC after deleting it already reuses the same instance, so its settings persist.)

- **About window** — new `AboutWindow.xaml(.cs)`, opened from a new "About RAT" button in `SettingsWindow`; fancy

credits to **Tobias & Claude** with helper **Christof**, plus a new `Icon.AboutRat` mascot vector.

- **Rat-themed DB/error popups** — new themed `RatDialog.xaml(.cs)` that shows a message beside a rat vector; new

`Icon.DatabaseError` (rat + database cylinder + warning). `ShowDbError` now uses it (`Icon.DatabaseError`), and

`ShowNoConnection` / `ShowActionFailed` were routed through it too (`Icon.NoConnection` / `Icon.ConnectionLost`).

Verified: builds (0 errors); screenshotted the About window (mascot + credits render) and the DB-error `RatDialog`

(database-rat graphic + themed card). Combo fix is structural (field→property); not exercised against a live backend.

Prompt 18 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Make sure the (JWT bearer) token gets renewed in the client.

Problem: `DatabaseConnection.Login()` fetched the JWT once and set it on the `HttpClient` forever. The backend

issues a short-lived token with no refresh endpoint, so once it expired every request returned 401 and the client was

stuck until a manual logout + re-login.

Changes made (AI region marked `prompt 18`, all in `RAT_Data/DatabaseConnection.cs`):

- Track the token + its expiry, parsed from the JWT `exp` claim (`ReadJwtExpiry`, `base64url-` decodes the payload; falls

back to a 5-minute lifetime if unparseable).

- `AuthenticateAsync(force)` re-runs the OAuth2 password login with the stored credentials and refreshes the bearer

token; serialised with a `SemaphoreSlim` so a burst of parallel requests triggers only one re-login.

- `EnsureAuthenticatedAsync()` proactively renews when the token is within a 30s margin of expiring.
- New central `SendAsync(requestFactory)` that every authenticated call now goes through: it ensures the token is

fresh first and, as a safety net, re-authenticates + retries once on a 401. Refactored

`GetJson` / `PostJson` and all

the `Put` / `Delete` / `Post` call sites (`NetworkObject`, interface, connection, permission, login, user, user-settings)

to build their request via a factory so the retry can reissue it. The `/user/login` call itself stays direct (it is the token endpoint). `Login()` now just calls `AuthenticateAsync` then reads `/user/me`.

Verified: builds (0 errors). The renewal path needs a live backend with an expiring token to exercise end-to-end,

so it wasn't runtime-tested here; the logic (proactive renew + reactive 401 retry, single-flight) is in place and the

existing login flow is unchanged on the happy path.

Prompt 19 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Some error popups still used the old system `MessageBox` (yellow-triangle), e.g. when adding a network

object to the database fails. Every such popup should use rat-themed vector art. Also the Logout vector had a rat tail

but no rat — give it a real rat.

Changes made (AI regions marked `prompt 19`):

- **Logout icon** (`Themes/Icons.xaml`) — replaced the lone tail with an actual little rat scurrying out the door

(body, head, ear, eye, nose, curled tail). Re-exported `doc/assets/vectorgraphics/Logout.png`.

- **All remaining** `MessageBox.Show` **calls converted to** `RatDialog` (themed popup with a rat vector). Files:

`Views/TopologyView.xaml.cs` (incl. the add-device DB error the user hit),

`ViewModels/TopologyViewModel.cs`,

`NetworkObject_UI/NetworkObjectSettingsWindow.xaml.cs` (SNMP + access control),

`NetworkObject_UI/LoginControl.xaml.cs`,

`SelectInterfaceWindow`, `UpdateInterfaceWindow`, `UpdateLoginWindow`, `InterfaceControl`,

`ManageUsersWindow`,

`EditUserWindow`, `SettingsWindow`, `EnterPasswordWindow`. Icon picked per case: `Icon.DatabaseError` for backend/DB

failures, `Icon.NoConnection` / `Icon.ConnectionLost` for connection issues, `Icon.LoginFailed` for permission/validation,

`Icon.Connected` / `Icon.LoginSuccess` for success, `Icon.Ethernet` for interface info. No `MessageBox.Show` remains.

Verified: builds (0 errors); re-rendered the Logout icon and confirmed the rat by screenshot. (RatDialog rendering

itself was verified in prompt 17.)

Prompt 20 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Implement a zoom feature in the user settings — a dropdown with multiple options, default 100%, min 50%

(zoomed out), max 300% (3x everything).

Changes made (AI regions marked prompt 20):

- `RAT_WPF/Themes/ZoomManager.cs` (new) — static app-wide zoom holder mirroring ThemeManager: `Levels`

(50/75/100/125/150/200/250/300), `Default=100`, `Min=50`, `Max=300`, `Apply(percent)` (clamped) + a `ZoomChanged`

event.

- `RAT_WPF/MainWindow.xaml(.cs)` — the shell `ContentControl` gets a `ScaleTransform` (via `LayoutTransform`) that the

code-behind updates on `ZoomManager.ZoomChanged`, so the whole app scales.

- `RAT_WPF/ViewModels/SettingsViewModel.cs` — `ZoomLevels` ("50%"..."300%") + `SelectedZoom`; selecting a level applies

it instantly via `ZoomManager` and best-effort persists it through

`IDatabaseConnection.EditUserSettings`.

- `RAT_WPF/SettingsWindow.xaml` — a **Zoom** dropdown added under Theme in the Appearance section.

Verified: builds (0 errors); screenshotted the Settings window (Zoom dropdown present) and the app forced to 200%

(logo, toolbar, sidebar + icons all rendered at 2x), confirming the scale transform zooms everything.

Notes / scope: there is no client-side *load* of saved user settings yet (no GetUserSettings wiring), so zoom

starts at 100% each run and changes are saved best-effort to the backend. `EditUserSettings` currently sends the whole

UserSettings, so it also writes the (unused) show_ports/show_interfaces defaults.

Prompt 21 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Document the RAT-Backend API (endpoints, example data, value limits like ">= 1"); add XML

`///` summaries describing the C# methods; create 10 xUnit tests; integrate abstract inheritance somewhere;

and document how to auto-generate a class diagram.

Changes made (AI regions marked `prompt 21`):

- **API docs** — `doc/markdown/API_documentation.md`: every endpoint of the FastAPI backend

(`C:\Users\chris\PycharmProjects\RAT-Backend`) with method, auth, example JSON, per-field limits (DB

VARCHAR sizes, `permissions` 0–4, token lifetime, etc.), the permission model, and the common error codes.

- **Abstract inheritance** — `RAT_Logic/DeviceDescriptor.cs` (new): an `abstract DeviceDescriptor` with

`abstract Type / IconKey / DisplayLabel`, a `virtual CanUseHostSpecs`, a `For(type)` factory, and five concrete subclasses (`PcDescriptor / Router / Switch / Server / Client`). `NetworkObject.Descriptor`

exposes it, and `Converters/IconKeyConverter` now resolves a device's icon polymorphically through it (instead of a local switch) — so the hierarchy is actually used, not dead code.

- **XML doc summaries** — added `<summary>` / `<param>` / `<returns>` / `<exception>` comments to the public

session + SNMP + host-info methods on `NetworkObject`
(`OpenSSH/OpenSSHstream/SendCommand/ExecuteSSH/`

SFTP+SCP

open/upload/download/list/Telnet/SetSnmp/GetSnmp/GetOwnDeviceInfos/GetOwnDeviceInterfaces)
and

on the new descriptor hierarchy. (Other public methods already had summaries from earlier prompts.)

- **10 xUnit tests** — `RAT_Tests/UnitTest1.cs` (project referenced `RAT_Logic` + added to the solution):

IP/port validation, the abstract `DeviceDescriptor` factory + polymorphic `CanUseHostSpecs`,
`NetworkObject.Descriptor`, and the access-rights rules (`GetRight` default-Hidden, global-admin
Owner,

`CanChangeRight` Admin-vs-Owner rules, `SetRight` / `ApplyRight` / `CanBeDeletedBy`). All 10 pass.

- **Class-diagram howto** — `doc/markdown/ClassDiagram.md`: VS Class Designer, Mermaid
`classDiagram`

(with a ready snapshot of the abstract hierarchy), PlantUML via `PlantUmlClassDiagramGenerator`, and
a

Roslyn/CI-automated route.

Verified: full solution builds (0 errors); `dotnet test` → 10/10 pass; app still launches.

Prompt 22 — Claude (model: Claude Opus 4.8, via Claude Code)

Request (TODOs): password length/criteria check in front- and backend; user editing; persist &
restore

zoom; persist `showInterfaces` + a toolbar toggle that draws each cable's interface name + IP; rolling
file

logging (folder, new `rat_<date>.tail` per start, keep 3); rewrite the API docs in German + prettier;
delete

a cable (`NetworkObjectConnection`) with the Delete tool; make the device icon non-blocking for the
delete/connection tools. Backend edits also marked `KI` + logged in the backend `AI_usage.md`.

Backend (`RAT-Backend`, marked `KI Claude <KI-13>`):

- `src/routers/user.py` — `validate_password()` (≥ 8 chars, ≥ 1 letter, ≥ 1 digit $\rightarrow$ HTTP 400); enforced in

`register()` and on password change in `edit_user()`. Documented in the backend `doc/markdown/AI_usage.md`.

Client (marked KI start/end (Claude Opus 4.8, prompt 22)):

- `RAT_Logic/PasswordPolicy.cs` (new) — shared client-side policy mirroring the backend; used by

`ManageUsersWindow` (create) and `EditUserWindow` (change). (User editing already existed — verified.)

- Zoom/settings: added `IDatabaseConnection.GetUserSettings()` (+ `DatabaseConnection/mock impl`); `LoginCommand`

restores zoom + showInterfaces after login; `SettingsViewModel` read-modify-writes so saving zoom keeps the

other settings.

- `RAT_WPF/Themes/DisplaySettings.cs` (new) — app-wide `ShowInterfaces` toggle + change event. Toolbar

checkbox in `TopologyView` flips it and persists it. `NetworkConnectionViewModel` exposes

`SourceLabel` / `TargetLabel` (name + IPv4) + label positions; the canvas draws them on each cable when on.

- Logging: `RAT_WPF/Logging/AppLogger.cs` (new) — `logs/` folder, a new `rat_<yyyy-MM-dd_HH-mm-ss>.tail`

per start, keeps the newest 3; `App.OnStartup` starts it + logs unhandled UI exceptions; login logs success/failure. (It already caught a real crash during a stress test.)

- Cable delete: `TopologyViewModel.DeleteConnectionFromCanvasAndDatabase`, `NetworkObjectDeleteCommand` now

also handles a `NetworkConnectionViewModel`, and a fat transparent hit-line on each cable triggers it when

the Delete tool is active.

- Tool clicks: `NetworkObjectView` — Cursor drags (MouseMove), Connection/Delete now act on a **click**

(`MouseLeftButtonUp`) of the whole node; the icon stays `IsHitTestVisible=False` so clicking it counts as

clicking the device.

Docs: `doc/markdown/API_documentation.md` rewritten in German with a nicer layout (tables, emojis, per-endpoint method/right/response tables, quick-reference limits incl. the new password policy).

Verified: full solution builds (0 errors); app launches; logging produces exactly 3 `rat_*.tail` files with the right naming.

Prompt 23 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Bug — a PC's interfaces existed only on the frontend. When a PC auto-populates this machine's

own interfaces, write them to the backend (together with the PC, which already saves its stats) on create.

Change (marked KI start/end (Claude Opus 4.8, prompt 23)):

- `RAT_WPF/Views/TopologyView.xaml.cs` — `PersistNewNetworkObject` now, right after `AddNetworkObject`

(which assigns the real backend id), loops the object's `NetworkInterfaces` and calls

`AddInterface(iface, model)` for each not-yet-saved one. So a dropped PC's auto-detected NICs (name + IPs)

are persisted server-side instead of living only in the client.

Verified: solution builds (0 errors).

Prompt 24 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Implement Telnet; make SSH and SFTP one login (entering SSH also covers SFTP and vice versa, since

same port/creds); if a cable exists from the user's PC to a device that has the matching login, auto-connect

before opening a shell / transferring; add a "use locally only" button on the login screen that skips login +

DB (nothing saved — use the mock, every op succeeds) with a reminder popup.

Changes made (AI regions marked KI start/end (Claude Opus 4.8, prompt 24)):

- **SSH/SFTP unified** — `RAT_Logic/Login.cs` `Covers(LoginType)` : an SSH or SFTP login serves both (SCP/Telnet

stay distinct). `NetworkObjectSettings.GetAllLoginsByType` uses it; new `FindLoginFor(type)` .

- **Telnet** — `RAT_Logic/NetworkObject.cs` : real raw-TCP Telnet
(`OpenTelnet` / `SendTelnet` / `ReadTelnet` / `CloseTelnet` ,

`IsTelnetConnected` , best-effort auto-login, IAC byte stripping); `IsConnected` now includes Telnet.
`LoginControl`'s

Connect button now opens Telnet (was "not implemented").

- **Auto-connect** — `NetworkObject.IsReachableFromHost()` + `EnsureConnected(type)` : if not connected, and the device

is reachable (a cable from the user's PC) and a covering login is stored, it opens the right session automatically

(SSH↔SFTP share the login). The settings window's shell-open, SSH command, and SFTP/SCP transfer buttons call it

(with clear popups when there's no cable or no login).

- **Local-only mode** — `RAT_WPF/Commands/LocalOnlyCommand.cs` (new) + a "Use locally only" button on `LoginView`

(`LoginViewModel.LocalOnlyCommand`): shows a reminder that nothing is saved, installs the rewritten

`DatabaseConnectionMock` as the active connection (every op just succeeds in memory, fake ids), seeds a local

user, and navigates to the topology. `RAT_Data/DatabaseConnectionMock.cs` rewritten from throw-stubs into a

working offline no-op connection.

Verified: solution builds (0 errors); 10/10 tests pass; the login screen shows the "Use locally only" button.

Prompt 25 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: cables clickable to edit type/name/speed/note (speed = standard-values dropdown) + deletable; a button

that auto-adds the user PC + discovers devices via nmap (add a Client per host, cable to existing devices, install

nmap on first run with a decline option that greys the button out); fix the infinite "add a connection first" trap on

the shell tab that locked the settings window; global admin can delete users + edit their flags/password/name.

Bug fix (the trap) — `NetworkObjectSettingsWindow.xaml.cs`: `SelectionChanged` bubbles, so switching the OUTER

settings tabs re-fired the inner ssh-tab handler with the "+" tab selected → auto-connect failed → popup → bounce →

loop forever. Fixed with `e.OriginalSource != SshShellsTabControl` guard + an `_openingShell` re-entrancy flag; shell

opening moved into `OpenNewShellTab()`.

Cables (marked prompt 25) — `RAT_Logic/ConnectionSpeed.cs` (new: Ethernet/Wi-Fi preset speeds);

`NetworkObject_UI/EditConnectionWindow.xaml(.cs)` (new: edit name/type/speed-dropdown/note); clicking a cable with the

Cursor tool edits it, with the Delete tool deletes it (`TopologyView.Connection_MouseLeftButtonUp`);

`IDatabaseConnection.EditConnection` (+ `DatabaseConnection.PUT` + mock);

`TopologyViewModel.PersistEditedConnection`.

Discovery (marked prompt 25) — `RAT_WPF/Discovery/NmapService.cs` (new: detect nmap, scan local /24, parse

hosts, auto-install from nmap.org); `TopologyViewModel.DiscoverDevicesAsync` (ensures the own-PC node, adds a Client +

interface per found IP not already modelled, cables existing-but-uncabled devices to the PC); "🔍 Discover devices"

topbar button, greyed out when nmap is missing/declined.

First-run setup (marked prompt 25) — `RAT_WPF/Setup/SetupService.cs` (new: desktop-shortcut via `WScript.Shell`,

remembers choices + the nmap-declined flag in marker files); `App.OnStartup` runs it once (offer shortcut + nmap

install; declining greys out Discover).

Admin user management — backend `DELETE /user/{id}` (KI-14, admin-only, cleans up dependent rows); client

`DatabaseConnection.DeleteUser` implemented; `ManageUsersWindow` got a per-row **Delete** button (with confirm,

can't delete yourself). Edit (name/password/admin/can_create) was already there via `EditUserWindow`.

Verified: solution builds (0 errors); 10/10 tests pass; app launches cleanly; backend `user.py` byte-compiles.

Notes / scope: the nmap scan + auto-install need a real machine/network to exercise end-to-end (not run here);

the logic compiles and the button correctly greys out when nmap is absent. Telnet has a session (Connect) but still no

dedicated Telnet *shell* tab — the shell tab remains SSH-only. Turning the app exe itself into a standalone installer

was done as in-app first-run setup (per the chosen option), not a separate installer executable.

Prompt 26 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: nmap discovery should add a **Switch** with enough interfaces when multiple devices are found (1:1

direct cable when only one); add a **Hub** network-object type (Switch already existed); make the existing backend

show_ports a toolbar option and integrate it with nmap; show friendly names for common ports ("22 - SSH",

"80 - HTTP", ...).

Changes made (marked KI start/end (Claude Opus 4.8, prompt 26)):

- **Hub type** — `RAT_Logic/NetworkObject.cs` `NetworkObjectType.Hub`; `DeviceDescriptor.cs` `HubDescriptor`

(reuses the switch icon). Backend stores `type` as a free string, so no backend change needed.

- **Port names** — `RAT_Logic/PortNames.cs` (new): `Describe(port)` → "22 - SSH" etc. for ~35 well-known ports.

- **nmap topology** — `Discovery/NmapService.cs`: `ScanLocalSubnetAsync(scanPorts)` now optionally runs `-F`

(fast port scan) and a line-based parser attaches each "x/tcp open" to its host (`DiscoveredHost.OpenPorts`).

`TopologyViewModel.DiscoverDevicesAsync` reworked: collects new hosts + existing-uncabled devices; **1 device →

direct PC↔device cable; multiple → a Switch with (N+1) ports, PC + every device wired to it** (`AddSwitchAsync`,

`CableViaSwitchAsync`). New clients carry their scanned open ports.

- **ShowPorts** — `Themes/DisplaySettings.cs` `ShowPorts` (+ change event), loaded/saved with the other user

settings (`LoginCommand`, `TopologyView.ShowPortsToggle_Click`); a "Show ports" checkbox in the topbar. Discovery

does a port scan only when it's on. `NetworkObjectInterface.OpenPorts` holds the ports; `NetworkObjectViewModel`

exposes `PortsText` (friendly names) shown on the device node when ShowPorts is on.

Verified: solution builds (0 errors); 10/10 tests pass; app launches with the new toolbar + node port panel.

Notes / scope: the actual nmap scan (ping + `-F` port scan) needs a real machine/network and wasn't run

end-to-end here — the parsing/topology logic compiles and is unit-reachable. Open ports are a discovery-time

annotation shown on the node; they aren't persisted to the backend (no column for them).

Prompt 27 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: improve nmap discovery and the canvas: fill in all available device info (subnet mask, OS, name) —

skip what nmap can't find; use **tracert** to identify the **router** (use a Router object) and add an **internet

Cloud **object behind it; name switch interfaces** Cisco-style** (GigabitEthernet0/1) instead of `port<n>`; fix the

"Show ports" view and show **all** open ports nmap finds, plus any port that has a ****login** configured but no nmap

response in red; **arrange discovered devices** in a circle **around the switch**; **make the canvas** pan left/right

with mouse drag + a horizontal scroll bar; **add** server detection** from open-port combinations (many service

ports e.g. 80+443+3306 → Server; SSH-only or HTTP-only → likely a switch, not a server); make the ****first-run** setup

run before the login window **and use the** RAT WPF style (RatDialog)** instead of the WindowsXP MessageBox.

Changes made (marked KI start/end (Claude Opus 4.8, prompt 27)):

- **Cloud type** — `RAT_Logic/NetworkObject.cs` `NetworkObjectType.Cloud`; `DeviceDescriptor.cs` `CloudDescriptor`

("Internet", `Icon.Cloud`); new rat-themed `Icon.Cloud` vector in `Themes/Icons.xaml`. Backend stores `type` as a

free string, so no backend change.

- **Device classifier** — `RAT_Logic/DeviceClassifier.cs` (new): `CiscoInterfaceName(i)` → `GigabitEthernet0/<i+1>`;

`ClassifyByPorts(openPorts)` — any DB port (MySQL/Postgres/...) or ≥ 2 server service ports ⇒ **Server**; SSH-only or

HTTP/HTTPS-only (or SSH+web-UI) ⇒ **Switch**; otherwise **Client**.

- **nmap richer info** — `Discovery/NmapService.cs`: `GetLocalSubnetInfo()` returns the real subnet mask + default

gateway; the port scan now also runs `-O` (OS detection) and the parser captures the OS guess; discovered hosts

get the local subnet mask stamped on. `FindRouterAsync()` / `ParseTracert()` run `tracert -d -h 5` to get the

first hop (the router) and whether the trace reached the internet (≥ 2 hops). `DiscoveredHost` gained `Os`,

`SubnetMask`.

- **Discovery topology** — `ViewModels/TopologyViewModel.cs`: `AddDiscoveredDeviceAsync` types each host via the

classifier (router forced from tracer) and fills name/OS/subnet mask/ports; `AddSwitchAsync` uses Cisco interface

names; `PlaceAround` lays the PC + devices on a **circle** around the switch;
`EnsureCloudBehindRouterAsync` adds

the **internet Cloud** wired to a spare router interface when the trace reached the internet; `CableAsync` now

picks a **free** interface on each end so the router's switch-cable isn't overwritten.

- **Ports view fix + red logins** — `ViewModels/NetworkObjectViewModel.cs` exposes `Ports` (a list of

`{ Text, IsUnreachableLogin }`) instead of a single string: it unions all scanned open ports with every configured-login port, and flags a login port that nmap did **not** see open.

`Views/NetworkObjectView.xaml`

renders each port on its own line via an `ItemsControl`; `Views/Converters/PortColorConverter.cs` paints the

unreachable-login ports **red** (others use the muted theme brush).

- **Canvas pan + scroll bar** — `Views/TopologyView.xaml` wraps the canvas in a `ScrollViewer`

(`HorizontalScrollBarVisibility="Auto"`) over a sized inner grid; `TopologyView.xaml.cs`

`PanCanvas_*` drag-pans

the view when the press starts on empty canvas (device drag / cable clicks are left alone via a visual-tree check).

- **First-run setup before login, themed** — `RatDialog.xaml(.cs)` gained a `Confirm(...)` Yes/No variant;

`App.xaml.cs` `OnStartup` is now async and **awaits** `RunFirstRunSetup()` **before building/showing the main window**,

using `RatDialog.Confirm` instead of `MessageBox`.

Tests: `RAT_Tests/UnitTest1.cs` — 6 new tests for `DeviceClassifier` (Cisco naming + the port heuristic:

SSH-only→Switch, HTTP-only→Switch, DB→Server, many services→Server, no ports→Client). 16/16 pass.

Verified: solution builds (0 errors); 16/16 tests pass. Per the user's request this was **not committed/pushed** —

left for the user to test first. The live nmap `-o` /tracert paths need a real network and admin rights for `-o` ;

the parsing/topology/classifier logic is unit-tested and compiles.

Prompt 28 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: ports still don't show (just a thin line under Settings); make the canvas effectively infinite when

panning but never let a device land in unreachable void; make the internet **Cloud** a draggable sidebar object like

PC/Client; add an **Access Point** object so a Wi-Fi host is modelled PC -(wifi)- AP - Switch - devices with the

Wi-Fi "cable" drawn **dashed**; review SNMP and fix bugs; make logging **much** more verbose; add a ****button** in

Settings to open the log folder**.

Changes made (marked KI start/end (Claude Opus 4.8, prompt 28)):

- **Ports display** — discovery now **always** runs the port scan (`ScanLocalSubnetAsync(scanPorts: true)`), so the

open-port data exists regardless of the "Show ports" toggle (previously the scan was skipped when the toggle was

off, so turning it on later showed nothing — the "thin line"). The node template already renders each port via an

`ItemsControl` ; configured-login ports nmap didn't see stay red.

- **Canvas** — `Views/CanvasLayout.cs` (new) defines the fixed canvas size (6000×4000) + `ClampX/ClampY` . The

`ScrollView` content grid was resized to match; device **drop, move** and the discovery **circle layout**

now clamp positions so a node can never be dragged/placed into unreachable void.

- **Cloud + Access Point as draggable objects** — new `NetworkObjectType.AccessPoint` (+ `AccessPointDescriptor` ,

rat-themed `Icon.AccessPoint`); both **Cloud** and **AccessPoint** are added to the sidebar `defaultItems` , so they

drag onto the canvas like PC/Client.

- **Wi-Fi topology + dashed link** — `Discovery/NmapService.IsActiveConnectionWireless()` detects a Wireless80211

uplink NIC. When wireless, discovery inserts an **AccessPoint** between the PC and the rest: PC -(wifi)-AP, then

the AP is the uplink into the Switch/single device.

`NetworkConnectionViewModel.IsWireless / StrokeDashArray`

drive a **dashed** cable Line (`CableWirelessAsync` creates a `NetworkConnectionType.Wireless` link).

- **SNMP fixes** — `NetworkObject.GetInterfaceInSameNetworkAsHost()` rewritten: it matched on subnet-**mask** string

equality (so any /24 device matched any /24 host NIC) and dereferenced possibly-null `IP / IPv4Mask` (NRE). Now it

compares real **network addresses** (ip & mask) and falls back to the first interface that has an IPv4. The three

SNMP methods null-check `IP` safely and use sane timeouts (8s GET/SET, 10s WALK; were 60s). The SNMP UI handlers

(`SnmpGet/Walk/Set_Click`) run **off the UI thread** (`Task.Run`) so the settings window no longer freezes, and

each attempt/outcome is logged.

- **Logging** — `AppLogger` gained `Debug` , `LogDirectory` , `OpenLogFolder()` . Added log lines across login

(attempt/success/settings-restore), logout, the whole discovery flow (scan result, per-host detail, tracer

router, topology decisions, each created device, done summary), device create/delete, cable delete, SNMP, and

startup phases.

- **Open log folder** — a "Diagnostics → Open log folder" button in `SettingsWindow` (`OpenLogFolder_Click`) opens

the `logs` folder in Explorer.

Tests: `RAT_Tests/UnitTest1.cs` — added a descriptor test for the new Cloud/AccessPoint types. **17/17 pass.**

Verified: solution + WPF project build (0 errors); 17/17 tests pass. As before this was **not committed/pushed** —

left for the user to test. The Wi-Fi/ `-o` /tracert paths still need a real network (and admin for `-o`) to exercise

end-to-end; the parsing/classifier/clamp logic is unit-tested and compiles.

Prompt 29 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: "Discover devices" takes forever because it port-scans the whole subnet up front; instead draw the

topology first (fast) and scan ports per-device afterwards in the background, only when "Show ports" is on, and only

use nmap if it's actually installed.

Changes made (marked KI start/end (Claude Opus 4.8, prompt 29)):

- **Fast topology, lazy ports** — `TopologyViewModel.DiscoverDevicesAsync` now draws the topology from a **ping-only**

sweep (`ScanLocalSubnetAsync(scanPorts: false)` , `-sn`), so the canvas appears quickly. After the draw, when

"Show ports" is on, `ScanPortsInBackgroundAsync` probes each discovered device **one at a time** and updates its

node (`vm.RefreshPorts()`) as results arrive — the user is never blocked on the slow `-F -O` work.

- **Per-host scan** — `Discovery/NmapService.ScanHostPortsAsync(ip)` runs `-F -O` against a single host and returns

its open ports + OS guess (reused parser).

- **nmap guard** — discovery aborts early (with a themed dialog + log) if `NmapService.IsInstalled()` is false, in

addition to the Discover button already being greyed out when nmap is missing/declined.

Verified: WPF project builds (0 errors); 17/17 tests pass. Committed, merged (if needed) and pushed per request.

Prompt 30 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: ports still don't show; the internet Cloud ends up outside the canvas on a fresh discovery (and rework

its icon); Wi-Fi / cable connections can't be deleted and there's no double-click to edit their type/speed/settings

(the earlier attempt didn't actually work); make the circle around the switch always big enough that nodes don't

overlap and stay inside the canvas; and make sure the touched code is MVVM — comment such ports as "ported to MVVM

by AI" rather than the usual KI style.

Changes made:

- **Cables clickable again (root cause)** — the devices' full transparent Canvas was layered ON TOP of the

connections layer, so every click (incl. over cables) hit the devices Canvas and the cable hit-lines never got

clicks. Reordered the canvas (`Views/TopologyView.xaml`): **devices on the bottom** (they own drag + drop, need a

transparent background for drops), **connections on top** with a `{x:Null}` panel background so empty areas still

pass clicks through to the devices/drop layer. Cable hit-line widened to 14px; labels made

`IsHitTestVisible=False`.

- **Cable delete + edit** — `Connection_MouseLeftButtonUp`: Delete tool deletes on a single click; Cursor tool opens

the editor on **double-click**; added `Connection_MouseRightButtonUp` so right-click edits with any tool. Edit

applies via the VM (`PersistEditedConnection`) and `NetworkConnectionViewModel.RefreshAfterEdit()` re-renders the

cable (solid ⇌ dashed) when the type changes. (`EditConnectionWindow` already had Name/Type/Speed/Note.)

- **Ports show now** — per-host scan (`NmapService.ScanHostPortsAsync`) split into a `-F` ports pass (never needs

admin, so ports always populate) + a SEPARATE best-effort `-0 --osscan-guess` OS pass that's ignored if it fails

(previously `-F -0` together could make nmap refuse the scan without admin → no ports). Turning the **Show ports**

toggle on now calls `TopologyViewModel.EnsurePortsScannedAsync()`, which scans every on-canvas device that has an

IP but no ports yet (so the toggle works even if discovery ran earlier with it off).

- **Cloud inside canvas + circle sizing** — `PlaceAround` reworked: the circle **radius grows with the device count**

(≥150px arc per node) so nodes never overlap, and the centre is shifted so the whole circle stays on the canvas

(no clamp pile-ups). The switch now starts near a visible centre. `EnsureCloudBehindRouterAsync` clamps the Cloud

onto the canvas (it could go negative-Y before).

- **Cloud icon reworked** — `Icon.Cloud` is now a clean cloud outline with a small globe (meridian + latitude lines)

instead of the tan blob with a rat head.

- **MVVM** — the new cable-interaction + placement code is routed through VM commands/methods; those ports are

commented "ported to MVVM by AI" per request. (Opening the edit dialog stays in the view, which is the correct

MVVM boundary for a window.)

Verified: WPF project builds (0 errors); 17/17 tests pass.

Prompt 32 — Claude (model: Claude Opus 4.8, via Claude Code)

Request: Anforderungstabelle in der Doku durch die vorgegebene Must-have/Nice-to-have-Liste

ersetzen; Hauptfenster im Vollbild starten und alle Fenster größer machen (Settings soll nicht

mehr scrollen müssen); ein lauffähiges `bin/` (exe + alle DLLs/Bilder) bereitstellen; der

Desktop-Verknüpfung/exe ein RAT-Icon geben; in Doku **und** README alle User-/Admin-Features und

das Vorgehen zum Starten (Installer) beschreiben; `API_documentation.md` gegen das Backend prüfen und aktualisieren; `requirements.txt` des Backends prüfen.

Changes made:

- **Doku:** `Dokumentation.md` — Abschnitt 2.2 durch die MH1–MH4 / NH1–NH6-Tabelle ersetzt (alle implementiert); neue Abschnitte 6.6 (Funktionsübersicht Anwender/Admin) und 6.7 (Installation/Start/Auslieferung). `Dokumentation.pdf` neu erzeugt.
- **README.md:** komplett überarbeitet — vollständige Feature-Liste (User + Admin), Installation (fertiges `bin/`, selbst bauen, Backend), Voraussetzungen, Doku-Links.
- **Fenster:** Hauptfenster `WindowState="Maximized"`; `Settings/EditUser/About/ManageUsers/Device` `Settings/EditConnection/UpdateLogin/UpdateInterface/SelectInterface` vergrößert bzw. `MaxHeight` erhöht, damit nicht standardmäßig gescrollt werden muss.
- **Icon:** `Assets/logo.ico` (aus `logo.png` generiert, mehrere Größen) + `<ApplicationIcon>` im `RAT_WPF.csproj`, damit exe/Taskbar/Desktop-Verknüpfung das RAT-Logo tragen.
- **bin/:** self-contained `dotnet publish -r win-x64 --self-contained` nach `bin/` (exe + alle DLLs + .NET-Laufzeit + eingebettete Bilder); per `git add -f` versioniert. `.gitattributes` ergänzt, damit Binärdateien nicht zeilenende-normalisiert werden. Zusätzliches Release-ZIP unter `dist/` (nicht im Git-Verlauf, dient als GitHub-Release-Asset).
- **API-Doku:** `API_documentation.md` aktualisiert — `DELETE /user/{id}` ist jetzt implementiert (war als „nicht implementiert“ markiert) inkl. Regeln/Aufräumen; `NetworkObject.type` um `Hub / Cloud / AccessPoint` ergänzt. Restliche Endpunkte gegen die Backend-Router geprüft — vollständig.
- **Backend** `requirements.txt`: strukturiert in direkte/transitive Abhängigkeiten; unpinned Pakete gepinnt (`python-jose==3.5.0`, `passlib==1.7.4`, `python-multipart==0.0.32`); ungenutzte Einträge entfernt (`django`, direktes `psutil` — `psutil` kommt transitiv über `fastapi-restful`). `pip install --dry-run` löst konfliktfrei auf.

Verified: Solution baut (0 Fehler); 17/17 Tests bestanden; `bin/RAT_WPF.exe` trägt das

RAT-Icon; PDF neu erzeugt.